Numpy提供了reshape方法用于改变数组的形状，reshape方法仅改变原始数据的形状，不改变原始数据的值。示例代码如下：

```
import numpy as np
arr = np.arange(12)  # 创建一维ndarray
arr1 = arr.reshape(3, 4)  # 设置ndarray的维度，改变其形态
```

执行结果如图2-12所示。

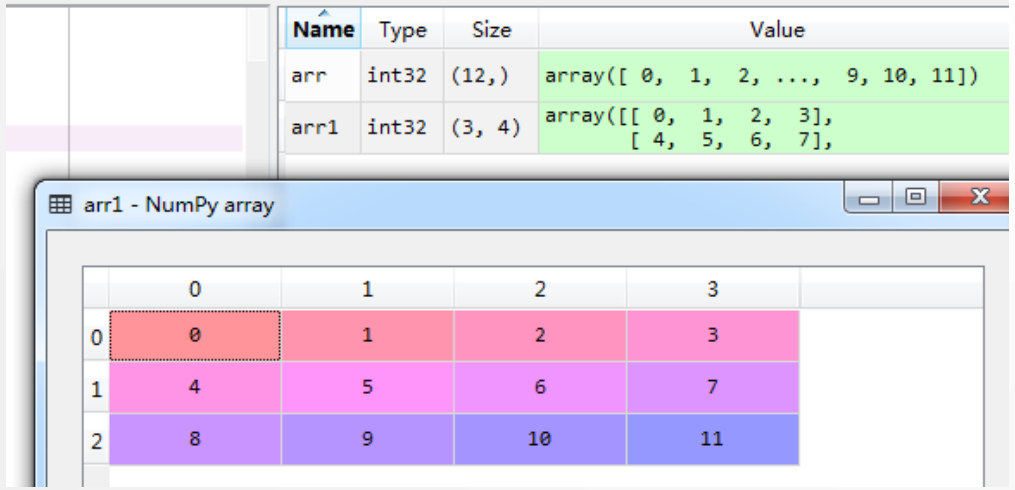



图2-12

以上示例代码是将一维数组形态变换为二维数组，事实上也可以将二维数组形态展平变换为一维数组，通过ravel()函数即可实现。示例代码如下：

```
import numpy as np
arr = np.arange(12).reshape(3, 4)
arr1=arr.ravel()
```

执行结果如图2-13所示。

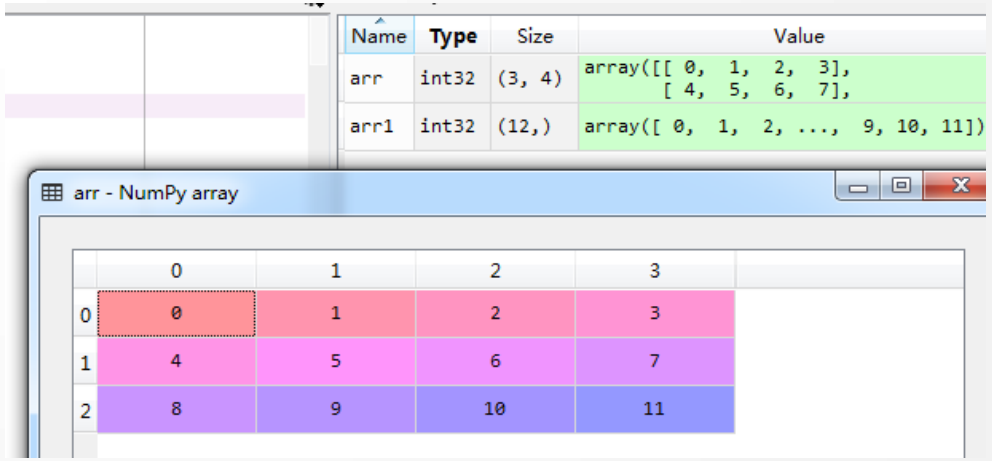



图2-13

# 2.9 数组排序与搜索



通过Numpyt提供的sort函数，可以对数组元素值按从小到大进行直接排序，示例代码如下：

```
import numpy as np
arr = np.array([5,2,3,3,1,9,8,6,7])
arr1=np.sort(arr)
```

执行结果如图2-14所示。

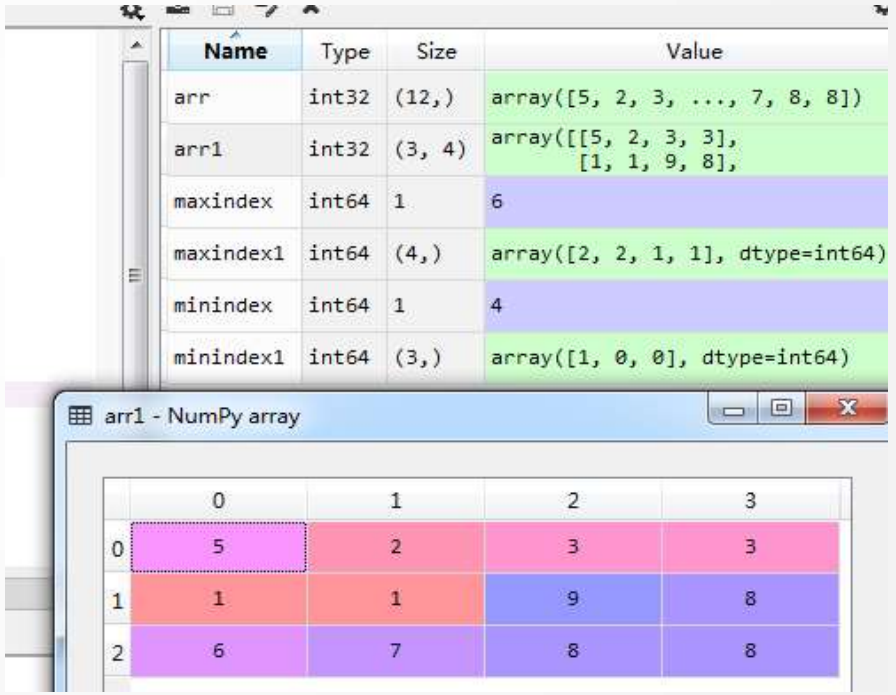
| Name | Type | Size | Value |
|---|---|---|---|
| arr | int32 | (9,) | array([5, 2, 3, 3, 1, 9, 8, 6, 7]) |
| arr1 | int32 | (9,) | array([1, 2, 3, 3, 5, 6, 7, 8, 9]) |

图2-14

通过Numpy提供的argmax和argmin函数，可以返回待搜索数组最大值和最小值元素的索引值，如果存在多个最大值或最小值，则返回第一次出现的索引。对于二维数组而已，可以通过设置axis=0或1返回各列或者各行最大值或最小值索引。需要注意的是，索引从0开始。示例代码如下：

```
import numpy as np
arr = np.array([5,2,3,3,1,1,9,8,6,7,8,8])
arr1=arr.reshape(3,4)
maxindex=np.argmax(arr)
minindex=np.argmin(arr)
maxindex1=np.argmax(arr1,axis=0)#返回各列最大值索引
minindex1=np.argmin(arr1,axis=1)#返回各行最小值索引
```

执行结果如图2-15所示。



| Name | Type | Size | Value |
|---|---|---|---|
| arr | int32 | (12,) | array([5, 2, 3, ..., 7, 8, 8]) |
| arr1 | int32 | (3, 4) | array([[5, 2, 3, 3],<br>[1, 1, 9, 8], |
| maxindex | int64 | 1 | 6 |
| maxindex1 | int64 | (4,) | array([2, 2, 1, 1], dtype=int64) |
| minindex | int64 | 1 | 4 |
| minindex1 | int64 | (3,) | array([1, 0, 0], dtype=int64) |

arr1 - NumPy array

| | 0 | 1 | 2 | 3 |
|---|---|---|---|---|
| 0 | 5 | 2 | 3 | 3 |
| 1 | 1 | 1 | 9 | 8 |
| 2 | 6 | 7 | 8 | 8 |

图2-15

Numpy的matrix是继承自NumPy的二维ndarray对象，不仅拥有二维ndarray的属性、方法与函数，还拥有诸多特有的属性与方法。同时，Numpy中的matrix和线性代数中的矩阵概念几乎完全相同，同样含有转置矩阵，共轭矩阵，逆矩阵等概念。

### 2.10.1 创建Numpy矩阵

Numpy中可使用mat、matrix或bmat函数来创建矩阵。使用mat函数创建矩阵时，若输入matrix或ndarray对象，则不会为它们创建副本。因此，调用mat函数与调用matrix(data, copy=False)等价，示例代码如下：

```
import numpy as np
mat1 = np.mat("1 2 3; 4 5 6; 7 8 9")
mat2 = np.matrix([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

执行结果如图2-16所示。

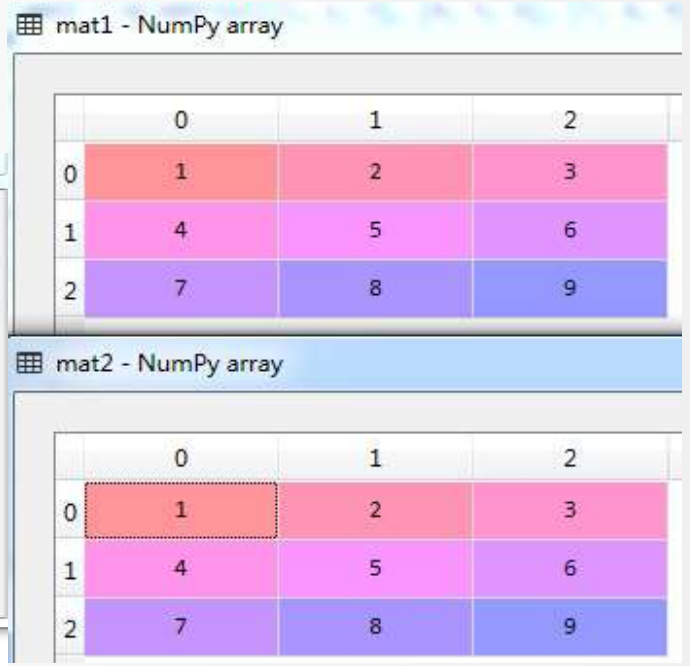



图2-16

在矩阵的日常使用过程中，将小矩阵组合成大矩阵是一种频率极高的操作。在Numpy中可以使用bmat分块矩阵函数实现，示例代码如下：

```
import numpy as np
arr1 = np.eye(3)
arr2 = 3*arr1
mat = np.bmat(“arr1 arr2; arr1 arr2”)
```

执行结果如图2-17所示。

图2-17

### 2.10.2 矩阵的属性和基本运算

矩阵有其特有的属性，如2-1所示。

表2-1 矩阵特有属性及其说明

| 属性 | 说明 |
|---|---|
| T | 返回自身的转置 |
| H | 返回自身的共轭转置 |
| I | 返回自身的逆矩阵 |

矩阵属性的具体查看方法，示例代码如下：

```
import numpy as np
mat = np.matrix(np.arange(4).reshape(2, 2))
mT=mat.T
mH=mat.H
mI=mat.I
```

执行结果如图2-18所示。

图2-18

在NumPy中，矩阵计算和ndarray计算类似，都能够作用于每个元素，比起使用for循环进行计算，在速度上更加高效，示例代码如下：

```
import numpy as np
mat1 = np.mat("1 2 3; 4 5 6; 7 8 9")
mat2 = mat1*3
mat3=mat1+mat2
mat4=mat1-mat2
mat5=mat1*mat2
mat6=np.multiply(mat1, mat2) #点乘
```

执行结果如图2-19所示。

图2-19

### 2.10.3 线性代数运算

线性代数是数学的一个重要分支。Numpy包含numpy.linalg模块，提供线性代数所需的功能，如计算逆矩阵、求解线性方程组、求特征值、奇异值分解以及求解行列式等。numpy.linalg模块中的一些常用函数表2-2所示。

表2-2 常用的numpy.linalg函数

| 函数名称 | 说明 |
|---|---|
| dot | 矩阵相乘 |
| inv | 求逆矩阵 |
| solve | 求解线性方程组 $Ax=b$ |
| eig | 求特征值和特征向量 |
| eigvals | 求特征值 |
| svd | 计算奇异值分解 |
| det | 求行列式 |

计算逆矩阵

在线性代数中，矩阵 与其逆矩阵 相乘得到一个单位矩阵 ，即 。使用numpy.linalg模块中的inv函数可以计算逆矩阵，示例代码如下：

```
import numpy as np
mat = np.mat('1 1 1; 1 2 3; 1 3 6')
inverse = np.linalg.inv(mat)
A=np.dot(mat, inverse)
```

执行结果如图2-20所示。

图2-20

## 2.求解线性方程组

矩阵可以对向量进行线性变换，这对应于数学中的线性方程组。numpy.linalg模块中的solve函数可以求解形如 的线性方程组，其中 为矩阵， 为一维或二维数组，是未知变量，示例代码如下：

```
import numpy as np
A = np.mat("1,-1,1; 2,1,0; 2,1,-1")
b = np.array([4, 3, -1])
x = np.linalg.solve(A, b)#线性方程组Ax=b的解
```

执行结果如图2-21所示。

图2-21

## 3.求解特征值与特征向量

设 是n阶方阵，如果存在数 和非零n维列向量 ，使得 成立，则称 是 的一个特征值，非零n维列向量 称为矩阵 的对应于特征值 的特征向量。numpy.linalg模块中的eigvals函数可以计算矩阵的特征值，eig函数可以返回一个包含特征值和对应的特征向量的元组，示例代码如下：

```
import numpy as np
A = np.matrix([[1, 0, 2], [0, 3, 0], [2, 0, 1]])
#A_value= np.linalg.eigvals(A)
A_value, A_vector = np.linalg.eig(A)
```

执行结果如图2-22所示

图2-22

4.奇异值分解

奇异值分解是线性代数中一种重要的矩阵分解，将一个矩阵分解为3个矩阵的乘积。numpy.linalg模块中的svd函数可以对矩阵进行奇异值分解，返回U、Sigma、V这3个矩阵，其中，U和V是正交矩阵，Sigma是一维，其元素为进行奇异值分解的矩阵的非零奇异值，可使用dig函数生成对角矩阵，示例代码如下：

```
import numpy as np
A = np.mat("4.0,11.0,14.0;  8.0,7.0,-2.0")
U, Sigma, V = np.linalg.svd(A, full_matrices=False)
```

执行结果如图2-23所示。

图2-23

5.计算矩阵行列式的值

矩阵行列式是指矩阵的全部元素构成的行列式，但构成行列式的矩阵为方阵时，行列式存在值。numpy.linalg模块中的det函数可以计算矩阵行列式的值，示例代码如下：

```
import numpy as np
A = np.mat("3,4; 5,6")
A_value=np.linalg.det(A)
```

执行结果如图2-24所示。

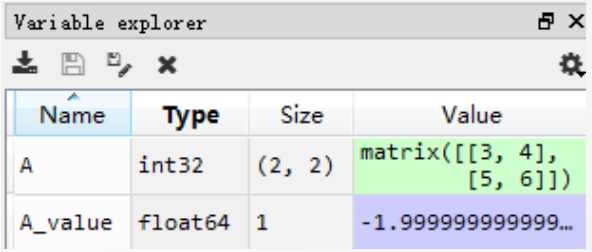



图2-24

# 本章小结



本章介绍了Python用于科学计算的基础包Numpy，包括如何导入并使用Numpy创建数组以及相关的数组运算、获得数组的尺寸、数组的四则运算与数学函数运算、数组的切片、数组连接和数据的存取、数组形态变换、数组元素的排序与搜索、矩阵及线性代数运算等相关知识。由于Numpy借鉴了Matlab矩阵开发思路，故Numpy的数组创建、运算、切片、连接及存取、排序与搜索、矩阵及线性代数运算均与Matlab的矩阵操作极为相似，如果读者具有一定的Matlab基础，可以将其对Matlab进行对比和细细体味，相信一定会有所收获。

**本章练习**

创建一个Python脚本，命名为test1.py，完成以下功能：

定义一个列表list1=[1,2,4,6,7,8]，将其转化为数组N1

定义一个元组tup1=(1,2,3,4,5,6)，将其转化为数组N2

利用内置函数，定义一个1行6列元素全为1的数组N3

将N1，N2，N3垂直连接，形成一个3行6列的二维数组N4

将N4保存为Python二进制数据文件（.npy格式）。

创建一个Python脚本，命名为test2.py，完成以下功能：

加载练习1中生成的Python二进制数据文件，获得数组N4

提取N4第1行中的第2、4个元素，第3行中的第1、5个元素，组成一个新的二维数组N5

将N5与练习1中的N1进行水平合并，生成一个新的二维数组N6。

3. 创建一个Python脚本，命名为test3.py，完成以下功能：

（1）生成两个2×2矩阵，并计算矩阵的乘积。

（2）求矩阵求矩阵 的特征值和特征向量。

（3）设有矩阵 ，试对其进行奇异分解。

（4）设有行列式 ，求其转置行列式 ，并计算 和 。